

Data Wrangling (Data Preprocessing)

Practical assessment 1

Tung Duc Nguyen, Hoang Nguyen

Setup

```
# Loading necessary packages:
library(kableExtra)
library(magrittr)
library(readxl)
library(dplyr)

##
## Attaching package: 'dplyr'

## The following object is masked from 'package:kableExtra':
##
##      group_rows

## The following objects are masked from 'package:stats':
##
##      filter, lag

## The following objects are masked from 'package:base':
##
##      intersect, setdiff, setequal, union

#citation("dplyr")
```

Student names, numbers and percentage of contributions

Table 1: Group information

Student name	Student number	Percentage of contribution
Tung Duc Nguyen	4275232	50
Hoang Nguyen	4280335	50

Data Description

Data set: Bail Justice and bail applications by outcome, age group and Indigenous status — data ending March 2026

Source: Crime Statistics Agency (CSA) Victoria, published via the Victorian Government's open data platform (<https://discover.data.vic.gov.au/dataset/bail-statistics>)

The data set records the outcomes of bail applications heard by a Bail Justice in Victoria, organized by age group and Indigenous status, reported as 12-month totals ending each March from 2017 to 2026.

Variables:

- Year – the year the 12-month reporting period ends
- Year ending – the month the period ends
- Data Source – the agency/court the data was recorded by
- Outcome – the result of the bail application (Bail granted, Bail refused, No application)
- Age Group – Adult 18 years and over, Child under 18 years, or Total
- Indigenous Status – Aboriginal and/or Torres Strait Islander, Non-Indigenous, or Total People
- Record Count – the number of people/applications matching that combination of Outcome, Age Group and Indigenous Status for that year

Read/Import Data

```
# Import the data
library(readxl)
bail_justice_data<-read_xlsx(
  "Bail_Justice_Data_Tables_Bail_Visualisation_Year_Ending_March_2026.xlsx",
  sheet = "Table 01")
class(bail_justice_data)
```

```
## [1] "tbl_df"      "tbl"        "data.frame"
```

First we ran the library(readxl) to load the package “readxl” into our session so we can use its function: “read_xlsx” to read our dataset from an excel file.

Then we ran “read_xlsx” to read our file “Bail_Justice_Data_Tables_Bail_Visualisation_Year_Ending_March_2026.xlsx” specifically sheet “Table 01” as that is where the data table is, while other sheets contain unusable data. We assigned the result to a new object named: bail_justice_data, which is now a dataframe in our R environment. To double check that fact, we also ran the class function on it.

Inspect and Understand

```
# Check the overall structure and object type of the dataset
str(bail_justice_data)
```

```
## tibble [159 x 7] (S3: tbl_df/tbl/data.frame)
##  $ Year          : num [1:159] 2026 2026 2026 2026 2026 ...
##  $ Year ending    : chr [1:159] "March" "March" "March" "March" ...
##  $ Data Source    : chr [1:159] "Bail Justice" "Bail Justice" "Bail Justice" "Bail Justice" ...
##  $ Outcome       : chr [1:159] "Bail granted" "Bail granted" "Bail granted" "Bail granted" ...
##  $ Age Group      : chr [1:159] "Adult 18 years and over" "Adult 18 years and over" "Adult 18 years and over" ...
##  $ Indigenous Status: chr [1:159] "Aboriginal and/or Torres Strait Islander" "Non-Indigenous" "Total" ...
##  $ Record Count   : num [1:159] 77 43 120 2 15 17 79 58 137 855 ...
```

```
class(bail_justice_data)
```

```
## [1] "tbl_df"      "tbl"        "data.frame"
```

Firstly for the inspection, we ran `str()` to get an overview of the dataset's structure, which shows us the total number of observations and variables, and for each variables it also shows the data type with a preview of the first few values. Additionally, we also ran `class()` to confirm that the dataset is a dataframe.

```
# Preview the first and last rows of the dataset
```

```
head(bail_justice_data)
```

```
## # A tibble: 6 x 7
##   Year 'Year ending' 'Data Source' Outcome      'Age Group' 'Indigenous Status'
##   <dbl> <chr>         <chr>         <chr>         <chr>         <chr>
## 1  2026 March        Bail Justice  Bail granted  Adult 18 y~  Aboriginal and/or ~
## 2  2026 March        Bail Justice  Bail granted  Adult 18 y~  Non-Indigenous
## 3  2026 March        Bail Justice  Bail granted  Adult 18 y~  Total People
## 4  2026 March        Bail Justice  Bail granted  Child unde~  Aboriginal and/or ~
## 5  2026 March        Bail Justice  Bail granted  Child unde~  Non-Indigenous
## 6  2026 March        Bail Justice  Bail granted  Child unde~  Total People
## # i 1 more variable: 'Record Count' <dbl>
```

```
head(bail_justice_data, n = 10)
```

```
## # A tibble: 10 x 7
##   Year 'Year ending' 'Data Source' Outcome      'Age Group' 'Indigenous Status'
##   <dbl> <chr>         <chr>         <chr>         <chr>         <chr>
## 1  2026 March        Bail Justice  Bail grant~  Adult 18 y~  Aboriginal and/or ~
## 2  2026 March        Bail Justice  Bail grant~  Adult 18 y~  Non-Indigenous
## 3  2026 March        Bail Justice  Bail grant~  Adult 18 y~  Total People
## 4  2026 March        Bail Justice  Bail grant~  Child unde~  Aboriginal and/or ~
## 5  2026 March        Bail Justice  Bail grant~  Child unde~  Non-Indigenous
## 6  2026 March        Bail Justice  Bail grant~  Child unde~  Total People
## 7  2026 March        Bail Justice  Bail grant~  Total        Aboriginal and/or ~
## 8  2026 March        Bail Justice  Bail grant~  Total        Non-Indigenous
## 9  2026 March        Bail Justice  Bail grant~  Total        Total People
## 10 2026 March        Bail Justice  Bail refus~  Adult 18 y~  Aboriginal and/or ~
## # i 1 more variable: 'Record Count' <dbl>
```

```
tail(bail_justice_data)
```

```
## # A tibble: 6 x 7
##   Year 'Year ending' 'Data Source' Outcome      'Age Group' 'Indigenous Status'
##   <dbl> <chr>         <chr>         <chr>         <chr>         <chr>
## 1  2017 March        Bail Justice  Bail granted  Adult 18 y~  Total People
## 2  2017 March        Bail Justice  Bail granted  Child unde~  Total People
## 3  2017 March        Bail Justice  Bail granted  Total        Total People
## 4  2017 March        Bail Justice  Bail refused  Adult 18 y~  Total People
## 5  2017 March        Bail Justice  Bail refused  Child unde~  Total People
## 6  2017 March        Bail Justice  Bail refused  Total        Total People
## # i 1 more variable: 'Record Count' <dbl>
```

We ran `head()` to preview the first 6 rows of our dataset by default and then ran it again with `n = 10` to view the first 10 rows for a slightly larger data sample. Afterwards we ran `tail()` to preview the last six rows of the dataset to check that both the head and tail had been imported correctly.

```
# Check for missing values
colSums(is.na(bail_justice_data))
```

```
##           Year      Year ending      Data Source      Outcome
##           0          0          0          0
##      Age Group Indigenous Status      Record Count
##           0          0          0          0
```

We used `colSums(is.na())` to check for missing values in each individual column. The table presented zero count of missing values so there are no missing values in the data set.

To achieve this, We referred to a GeeksforGeeks tutorial on finding and counting missing values in R to confirm the correct functions to use for this check (<https://www.geeksforgeeks.org/r-language/how-to-find-and-count-missing-values-in-r-dataframe/>)

```
# Check the data type of individual variables
class(bail_justice_data$Outcome)
```

```
## [1] "character"
```

```
class(bail_justice_data$`Record Count`)
```

```
## [1] "numeric"
```

```
typeof(bail_justice_data$Year)
```

```
## [1] "double"
```

```
typeof(bail_justice_data$`Record Count`)
```

```
## [1] "double"
```

```
is.numeric(bail_justice_data$Year)
```

```
## [1] TRUE
```

```
is.numeric(bail_justice_data$`Record Count`)
```

```
## [1] TRUE
```

```
is.numeric(bail_justice_data$`Data Source`)
```

```
## [1] FALSE
```

We ran `class()` on the Outcome and Record Count columns specifically to check the data type of each and then used `typeof()` on Year and Record Count to check how R is storing these numeric variables. Finally, we used `is.numeric()` to explicitly test whether Year, Record Count, and Data Source are numeric variables. This confirmed that Year and Record Count are numeric (returning TRUE), while Data Source is not (returning FALSE), since it's a text variable rather than a numeric one.

```
# Check the attributes (metadata) of the dataset
attributes(bail_justice_data)
```

```
## $names
## [1] "Year"           "Year ending"    "Data Source"
## [4] "Outcome"        "Age Group"      "Indigenous Status"
## [7] "Record Count"
##
## $row.names
## [1] 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18
## [19] 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36
## [37] 37 38 39 40 41 42 43 44 45 46 47 48 49 50 51 52 53 54
## [55] 55 56 57 58 59 60 61 62 63 64 65 66 67 68 69 70 71 72
## [73] 73 74 75 76 77 78 79 80 81 82 83 84 85 86 87 88 89 90
## [91] 91 92 93 94 95 96 97 98 99 100 101 102 103 104 105 106 107 108
## [109] 109 110 111 112 113 114 115 116 117 118 119 120 121 122 123 124 125 126
## [127] 127 128 129 130 131 132 133 134 135 136 137 138 139 140 141 142 143 144
## [145] 145 146 147 148 149 150 151 152 153 154 155 156 157 158 159
##
## $class
## [1] "tbl_df"      "tbl"        "data.frame"
```

We ran `attributes()` on our data frame to show the column names, the row names, and the class of the object, which is useful for confirming the overall shape and labeling of the dataset beyond just the individual variable types.

```
# Convert categorical variables from character to factor
bail_justice_data[c("Outcome",
"Age Group", "Indigenous Status", "Data Source")]<-lapply(bail_justice_data[c("Outcome",
"Age Group", "Indigenous Status", "Data Source")], as.factor)
str(bail_justice_data)
```

```
## tibble [159 x 7] (S3: tbl_df/tbl/data.frame)
## $ Year      : num [1:159] 2026 2026 2026 2026 2026 ...
## $ Year ending : chr [1:159] "March" "March" "March" "March" ...
## $ Data Source : Factor w/ 1 level "Bail Justice": 1 1 1 1 1 1 1 1 1 1 ...
## $ Outcome    : Factor w/ 3 levels "Bail granted",...: 1 1 1 1 1 1 1 1 2 ...
## $ Age Group  : Factor w/ 3 levels "Adult 18 years and over",...: 1 1 1 2 2 2 3 3 3 1 ...
## $ Indigenous Status: Factor w/ 3 levels "Aboriginal and/or Torres Strait Islander",...: 1 2 3 1 2 3 ...
## $ Record Count : num [1:159] 77 43 120 2 15 17 79 58 137 855 ...
```

```
levels(bail_justice_data$`Age Group`)
```

```
## [1] "Adult 18 years and over" "Child under 18 years"
## [3] "Total"
```

Since Outcome, Age Group, Indigenous Status, and Data Source are all categorical variables that were imported as character type, we used lapply() to apply the as.factor() function to all four columns and reassign them back into the data frame. This converts each of them from character to factor, which is more appropriate for categorical data. we then ran str() again to confirm the four columns are now listed as factors instead of characters, and ran levels() on the Age Group column specifically to check what categories exist now that it has been converted, confirming the conversion worked.

```
# Convert Year from numeric to integer
bail_justice_data$Year <- as.integer(bail_justice_data$Year)
bail_justice_data$`Record Count` <- as.integer(bail_justice_data$`Record Count`)
typeof(bail_justice_data$Year)
```

```
## [1] "integer"
```

```
typeof(bail_justice_data$`Record Count`)
```

```
## [1] "integer"
```

Finally, we converted the Year column and the Record Count column from numeric (double) to integer using as.integer(), since they are really whole-numbers rather than continuous measurements, therefore integer is the more appropriate and accurate type for them we then ran typeof() again to confirm the conversions were successful and it now shows as “integer” rather than “double”.

Summary Statistics

```
#Filter out "Total" (Age Group) and "Total People" (Indigenous Status)
bail_justice_data <- bail_justice_data %>%
filter(`Age Group` != "Total", `Indigenous Status` != "Total People")
summary_stats <- bail_justice_data %>%
  group_by(Outcome) %>%
  summarise(
    Mean = mean(`Record Count`, na.rm = TRUE),
    Median = median(`Record Count`, na.rm = TRUE),
    Q1 = quantile(`Record Count`, 0.25, na.rm = TRUE),
    Q3 = quantile(`Record Count`, 0.75, na.rm = TRUE),
    SD = sd(`Record Count`, na.rm = TRUE)
  )
summary_stats
```

```
## # A tibble: 3 x 6
##   Outcome      Mean Median    Q1    Q3    SD
##   <fct>      <dbl> <dbl> <dbl> <dbl> <dbl>
## 1 Bail granted   63.0   56   33.5  100.  44.7
## 2 Bail refused  480.  453  131.  788. 385.
## 3 No application 73.1   50.5  24.8  120.  63.9
```

Before generating summary statistics, we filtered out the ‘Total’ (Age Group) and ‘Total People’ (Indigenous Status) rows, as they are subtotals of the other categories rather than independent values, and including them would double-count people and mix these less detailed value into the calculation and reduces the quality of our data.

To start, we grouped the bail justice data by the categorical variable “Outcome” and calculated the summary statistics for the numeric variable “Record Count”. Then, We used `mean()` to find the average record count, `median()` to find the middle value, `quantile()` to calculate the first and third quartiles, and `sd()` to find the standard deviation for each outcome. Additionally, We also used `na.rm = TRUE` so any missing values would be ignored during the calculations.

From the output, we found that ‘Bail refused’ has the highest mean Record Count of approximately 479.92 and also the highest median of 453, while having the highest standard deviation of approximately 384.78, all of which shows that the Record Count for Bail refused varies considerably more than the other outcomes. Additionally, ‘Bail granted’ has the lowest mean Record Count of approximately 62.96 and the lowest standard deviation of approximately 44.66.”

Subsetting

```
# Subset data and convert it to a matrix
bail_subset <- bail_justice_data[1:10, ]
bail_subset
```

```
## # A tibble: 10 x 7
##   Year 'Year ending' 'Data Source' Outcome      'Age Group' 'Indigenous Status'
##   <int> <chr>         <fct>         <fct>         <fct>         <fct>
## 1  2026 March          Bail Justice  Bail grant~ Adult 18 y~ Aboriginal and/or ~
## 2  2026 March          Bail Justice  Bail grant~ Adult 18 y~ Non-Indigenous
## 3  2026 March          Bail Justice  Bail grant~ Child unde~ Aboriginal and/or ~
## 4  2026 March          Bail Justice  Bail grant~ Child unde~ Non-Indigenous
## 5  2026 March          Bail Justice  Bail refus~ Adult 18 y~ Aboriginal and/or ~
## 6  2026 March          Bail Justice  Bail refus~ Adult 18 y~ Non-Indigenous
## 7  2026 March          Bail Justice  Bail refus~ Child unde~ Aboriginal and/or ~
## 8  2026 March          Bail Justice  Bail refus~ Child unde~ Non-Indigenous
## 9  2026 March          Bail Justice  No applica~ Adult 18 y~ Aboriginal and/or ~
## 10 2026 March          Bail Justice  No applica~ Adult 18 y~ Non-Indigenous
## # i 1 more variable: 'Record Count' <int>
```

```
bail_matrix <- as.matrix(bail_subset)
bail_matrix
```

```
##      Year  Year ending Data Source  Outcome
## [1,] "2026" "March"      "Bail Justice" "Bail granted"
## [2,] "2026" "March"      "Bail Justice" "Bail granted"
## [3,] "2026" "March"      "Bail Justice" "Bail granted"
## [4,] "2026" "March"      "Bail Justice" "Bail granted"
## [5,] "2026" "March"      "Bail Justice" "Bail refused"
## [6,] "2026" "March"      "Bail Justice" "Bail refused"
## [7,] "2026" "March"      "Bail Justice" "Bail refused"
## [8,] "2026" "March"      "Bail Justice" "Bail refused"
## [9,] "2026" "March"      "Bail Justice" "No application"
## [10,] "2026" "March"      "Bail Justice" "No application"
##      Age Group      Indigenous Status
## [1,] "Adult 18 years and over" "Aboriginal and/or Torres Strait Islander"
## [2,] "Adult 18 years and over" "Non-Indigenous"
## [3,] "Child under 18 years"    "Aboriginal and/or Torres Strait Islander"
```

```
## [4,] "Child under 18 years"      "Non-Indigenous"
## [5,] "Adult 18 years and over"  "Aboriginal and/or Torres Strait Islander"
## [6,] "Adult 18 years and over"  "Non-Indigenous"
## [7,] "Child under 18 years"      "Aboriginal and/or Torres Strait Islander"
## [8,] "Child under 18 years"      "Non-Indigenous"
## [9,] "Adult 18 years and over"  "Aboriginal and/or Torres Strait Islander"
## [10,] "Adult 18 years and over"  "Non-Indigenous"
##      Record Count
## [1,] " 77"
## [2,] " 43"
## [3,] "  2"
## [4,] " 15"
## [5,] "855"
## [6,] "548"
## [7,] " 41"
## [8,] "141"
## [9,] "185"
## [10,] "112"
```

```
str(bail_matrix)
```

```
## chr [1:10, 1:7] "2026" "2026" "2026" "2026" "2026" "2026" "2026" "2026" ...
## - attr(*, "dimnames")=List of 2
## ..$ : NULL
## ..$ : chr [1:7] "Year" "Year ending" "Data Source" "Outcome" ...
```

```
class(bail_matrix)
```

```
## [1] "matrix" "array"
```

```
typeof(bail_matrix)
```

```
## [1] "character"
```

Firstly, we used `[1:10, ]` to subset the first 10 observations from the bail justice data while keeping all variables, then we used `as.matrix()` to convert the subsetted data frame into a matrix.

From the output, we found that the matrix was a character matrix. This happened because a matrix can only contain one data type, while a data frame can contain different data types in different columns. Therefore, since the subset contains categorical variables such as “Outcome”, “Age Group” and “Indigenous Status” together with numeric variables such as “Year” and “Record Count”, R converts all values to character so that the matrix can store them using one common data type.

Create a new Data Frame

```
# Create a new data frame, provide R codes here.
student_id <- 1:10
satisfaction <- c("Low", "Medium", "High", "Medium", "Low", "High", "Medium", "High", "Low", "Medium")
satisfaction <- factor(satisfaction, levels = c("Low", "Medium", "High"), ordered = TRUE)
new_data <- data.frame(student_id, satisfaction)
new_data
```



```
##      student_id satisfaction
## 1           1           Low
## 2           2          Medium
## 3           3           High
## 4           4          Medium
## 5           5           Low
## 6           6           High
## 7           7          Medium
## 8           8           High
## 9           9           Low
## 10          10          Medium
```

First, we created two vectors called "student_id" and "satisfaction" with 10 observations:

- The student_id variable contains integer values from 1 to 10.

- The satisfaction variable contains three ordinal categories: "Low", "Medium" and "High".

We used factor() with ordered = TRUE to convert satisfaction into an ordered factor and set the levels:

```
''' r
# Check the structure and ordinal levels
str(new_data)

## 'data.frame':   10 obs. of  2 variables:
## $ student_id : int  1 2 3 4 5 6 7 8 9 10
## $ satisfaction: Ord.factor w/ 3 levels "Low"<"Medium"<..: 1 2 3 2 1 3 2 3 1 2
```

```
levels(new_data$satisfaction)
```

```
## [1] "Low"      "Medium"   "High"
```

Next we used str() to check the structure of the new data frame and levels() to check that the satisfaction variable was an ordered factor with the levels in the correct order.

```
# Create and add a numeric variable
hours_studied <- c(8, 12, 15, 10, 6, 18, 11, 14, 7, 9)
new_data <- cbind(new_data, hours_studied)
new_data
```

```
##      student_id satisfaction hours_studied
## 1           1           Low             8
## 2           2          Medium            12
## 3           3           High            15
## 4           4          Medium            10
## 5           5           Low             6
## 6           6           High            18
## 7           7          Medium            11
## 8           8           High            14
## 9           9           Low             7
## 10          10          Medium            9
```

```
str(new_data)
```

```
## 'data.frame':   10 obs. of  3 variables:
## $ student_id   : int   1 2 3 4 5 6 7 8 9 10
## $ satisfaction : Ord.factor w/ 3 levels "Low"<"Medium"<...: 1 2 3 2 1 3 2 3 1 2
## $ hours_studied: num   8 12 15 10 6 18 11 14 7 9
```

Finally, we created another numeric vector called “hours_studied” with 10 observations and used `cbind()` to add it to the existing data frame, which outputed the final data frame containing 10 observations and 3 variables.

References

Bache SM & Wickham H (2026) *Magrittr*: A forward-pipe operator for R,

Crime statistics Agency Data tables - bail statistics - DataVic (2026), Vic.gov.au, accessed August 17, 2026. <https://discover.data.vic.gov.au/dataset/bail-statistics>

GeeksforGeeks (2022) How to find and count missing values in R DataFrame, accessed August 17, 2026. <https://www.geeksforgeeks.org/r-language/how-to-find-and-count-missing-values-in-r-dataframe/>

TUNG1205 (2017) GitHub - TUNG1205/bail-outcomes-in-victoria-age-and-indigenous-status-trends: This project analyses bail application outcomes recorded by Victoria’s bail justices from 2017 to 2026, broken down by age group and indigenous status It examines how bail decisions vary across these categories over a decade, using record count as the key numeric measure, GitHub, accessed August 20, 2026. <https://github.com/TUNG1205/Bail-Outcomes-in-Victoria--Age-and-Indigenous-Status-Trends>

Wickham H & Bryan J (2026) *READXL*: Read excel files, CRAN Package Documentation

Wickham H, François R, Henry L, Müller K & Vaughan D (2026) *DPLYR*: A grammar of data manipulation,

Zhu H (2026) *kableExtra*: Construct complex table with “Kable” and pipe syntax,

Presentation

Tung’s Presentation

link: <https://rmit-arc.instructuremedia.com/embed/d76b8ee7-93f6-4c7a-8230-d336d4d8617c>

Hoang’s Presentation

link: <https://rmit-arc.instructuremedia.com/embed/a7c0b929-3974-4bcb-80c9-3f5f961197b7>